Motion Planning for Multi-DOF Robotic Manipulators: A Direct Optimal Control Approach

April 17, 2024

Catherine Guo
University of Toronto Institute for Aerospace Studies

Andres Cervera Rozo
University of Toronto Institute for Aerospace Studies

Ketan Vasudeva
University of Toronto Institute for Aerospace Studies

1. Introduction

Robotic manipulators have been successfully deployed in many different industries, from automotive assembly factories to space exploration. Their hardware enables them to handle large, heavy payloads at high speeds, and operate in environments that may be dangerous or unsuitable for human workers. The motion planning problem associated with manipulators is to find a sequence of actuator inputs, typically joint torques, that will bring the end effector along a set of desired waypoints, in order to complete some task. This is made difficult by the need to convert between Cartesian task space, which is most commonly used to describe desired end effector poses and obstacle locations, and configuration space, which describes the manipulator pose using a minimal set of manipulator parameters, typically joint angles. The challenge is further exacerbated by the high dimensionality of the problem – while most mobile robotic motion planning operations can be expressed in 2-dimensional spaces, the task space for manipulators is 3-dimensional, and the configuration space depends on the number of degrees of freedom, often 6 or more.

Due to the versatility of robotic manipulators and their widely varying applications, it is often desirable to generate trajectories that are optimal in different regards. For example, in industrial manufacturing settings, it may be ideal to minimize the time needed to complete a certain maneuver. By contrast, in space environments, it might be more important to optimize with respect to power consumption or energy usage. These types of manipulator motion planning problems can be considered to be constrained trajectory optimization problems. Trajectory optimization is a common application of optimal control theory, which deals with the problem of finding the best control strategy for bringing a given dynamical system from its initial state to some desired goal state, where “best” is judged according to some predetermined performance criterion. This performance criterion is referred to as the objective function. Most classical optimal control techniques rely on gradient-based computational methods that iteratively converge to the optimal solution. Genetic algorithms are modern optimization techniques that incorporate heuristics inspired by biological natural selection processes to search for an optimal solution.

The purpose of this project is to explore the use of numerical trajectory optimization techniques in generating optimal, feasible trajectories for a two-link, two-degree-of-freedom robotic manipulator. The performance of an interior-point optimization algorithm will be tested under two different objective functions, one being minimum-power, and the other being minimum-time. Additionally, an equivalent genetic algorithm variation will be explored, and its suitability for the trajectory optimization case will be evaluated.

2. Background

2.1 Motivation

The main challenge associated with motion planning for robotic manipulation problems is the highly constrained nature of the problem, due to the chained manipulator kinematics. A popular and intuitive approach is to find a path from the start to the goal through Cartesian space, and then use inverse kinematics to solve for the required joint angles at a series of waypoints [1]. The space between these waypoints can then be filled in by interpolating the joint angles. However, this method is unreliable for large movements which require significant changes in the manipulator configuration. Finding a feasible path through Cartesian space does not mean it is a feasible path for the manipulator, and a shortest path through space rarely correlates to a minimal effort path for the manipulator's actuators.

One way of handling this is to perform the motion planning entirely in the manipulator's configuration space. This is the approach taken by Kuffner and LaValle., in [2], which uses rapidly exploring random trees to search through a manipulator's high-dimensional configuration space to find a minimal cost path. A key drawback to this approach is the need to perform some sort of post-processing to smooth the final path, as well as the need to encode all the obstacles and environment variables in configuration space.

Another alternative is to use kinematic trajectory optimization, whereby the entire sequence of joint angles / torques is directly optimized over with respect to some objective function [1]. This is referred to as direct optimal control, and it poses several major advantages:

1. The given solution is truly and wholly optimal with respect to the real environment, and does not require post-processing or smoothing.
2. The objective function is interchangeable and highly customizable. Multiple objective functions can even be combined to obtain trajectories that are optimal in multiple ways.
3. The optimal trajectory can be solved for directly in terms of the control inputs, avoiding the need for an analytic derivation of the manipulator's inverse kinematics.

2.2 History of Optimal Control Theory

The motion planning problem for robotic manipulators can be thought of as a "point-to-point" optimal control problem, whereby the goal is to transfer the system from its initial state to a given goal state, while adhering to dynamic constraints and attaining minimal cost [3]. The underlying mathematics of solving such point-to-point optimal control problems has a deep history, and has been studied by many of the most recognized names in mathematics. It originated as an extension of the calculus of variations, which was invented by Isaac Newton in order to solve for the minimum drag nose profile of a projectile. This was extended in 1696, when Bernouilli issued a challenge to his fellow mathematicians to find a method of obtaining the minimum-time orbital path of a movable point M from point A to point B. Amongst those who submitted solutions were Leibniz, l'Hopital, and Newton. This triggered a flurry of activity to solve similar problems, and contributions were made soon after by Euler, Lagrange, Legendre, and more [4].

The modern era of optimal control theory is based on two main ideas: dynamic programming, and Pontryagin's maximum principle [5]. Dynamic programming relies on the Bellman optimality principle, introduced by Richard E. Bellman in 1952, which is that for a given state-action sequence, the removal of the first state-action pair will still yield an optimal sequence. Thus, a complex optimization problem can be solved by tackling a set of simpler sub-problems. This philosophy is the basis for many classical optimal control methods. However, such approaches often yield intractable problems due to the curse of dimensionality. The Pontryagin maximum principle was formulated soon after, in 1956, by Lev Pontryagin of the Soviet Union. It provides a necessary condition for an optimal trajectory, which is that the optimal path must solve the associated Hamiltonian, a boundary-value ordinary differential equation. These two developments have led to many computationally feasible gradient-based optimization techniques, that can be used to directly optimize entire trajectories with respect to a given objective function.

2.3 Generic Trajectory Optimization Problems

A general constrained optimization problem can be expressed in the following form, which we will call the primal problem (1):

$$\min_{x\in\Omega} f(x) \tag{1}$$

$$\text{Subject to } h_i(x) = 0, \quad i = 1, \dots, m$$
$$g_j(x) \leq 0, \quad j = 1, \dots, r$$

When *f(x)* is a nonlinear function is referred to as a nonlinear programming problem (NLP). This is typically the case for trajectory optimization, where the problem becomes:

$$\min_{x(t),u(t)} J\big(u(t), x(t)\big) \tag{2}$$

$$\text{Subject to: } \dot{x}(t) = f\big(x(t), u(t)\big)$$
$$x(0) = x_0$$
$$x(T) = x_T$$
$$u_{min} \leq u(t) \leq u_{max}$$
$$x_{min} \leq x(t) \leq x_{max}$$

The state and control input variables are the decision variables to be optimized over, and are denoted by *x(t)* and *u(t)* respectively. The objective, denoted by *J(u, x)*, can be chosen depending on the problem at hand. In this project, we will look at minimum energy, minimum time, and minimum torque cases. The system dynamics are imposed by the $\dot{x}(t)$ equality constraints, and the initial and final conditions are imposed by the $x(0), x(T)$ equality constraints. The actuator and state limitations are handled via inequality constraints.

An important concept in optimization is that of duality. Duality is the principle that any minimizing optimization problem has a corresponding and equivalent maximization problem, and vice versa. In this project, for the sake of consistency, we will only consider optimization problems that aim to minimize a cost function. This original minimization problem is thus referred to as the primal problem, and the corresponding maximization problem is the dual problem. The function being optimized for in the dual problem is the Lagrangian of the primal problem, denoted below as $L(x, u, v)$ and the optimization variables are called the dual variables, denoted below by u, v.

$$L(x, u, v) = f(x) + \sum_{i=1}^{m} u_i g_i(x) + \sum_{j=1}^{r} v_j l_j(x)$$

$$\max_{u,v}(\inf_{x} L(x, u, v)) \quad subject\ to \quad u \geq 0 \tag{3}$$

The implication of duality is that the primal problem can be solved by finding an upper bound on the solution to the associated dual problem (3). This fact is leveraged in many optimization algorithms, which tackle the primal and dual problems simultaneously [6][7].

2.4 Interior Point Optimization

Nonlinear programming problems are difficult to solve due to their non-convexity, which means that local minima or maxima tend to be present. The optimization tool used for this project was the IPopt solver in Casadi. IPopt is a primal-dual interior-point line search method, and was developed in 2005 in the seminal work released by Wachter and Biegler [8]. It works by converting the constrained NLP into an unconstrained problem, by penalizing constraint violations with a barrier function. An initial guess is made, which must fall within the feasible set, and the algorithm then iterates towards the optimal solution along what is called the “central path”. All points along the central path fall within the feasible set, which is where the term “interior point” comes from.

The unconstrained problem, which can be referred to as the barrier problem, has an associated set of first-order necessary conditions that must be met for any solution to be optimal. These conditions are called the Karush-Kuhn-Tucker (KKT) conditions, and they impose requirements on both the primal problem as well as the associated dual problem. These conditions, which are referred to as stationarity, primal feasibility, dual feasibility, and complementary, are a set of nonlinear equations which must all be satisfied for an optimal solution. Interior point methods use Newton's method to find a set of primal and dual variables, (x^*, u^*, v^*), which will satisfy these equations [6].

2.5 Overview of Genetic Algorithms

Evolutionary algorithms (EAs) are a set of heuristic-based optimization algorithms that leverage the concept of Darwinian evolution to evolve artificial agents/representations [10]. Evolutionary algorithms can be broken down into four main steps:

1. **Initialization:** the first set of candidate solutions are produced, often through random generation.
2. **Selection:** candidate solutions are evaluated based on a fitness function. A fitness function outputs a value indicating the performance of the candidate solutions.
3. **Genetic Operators:** the genetic operators are inspired from natural selection and are used to create new candidate solutions. Typically, there are two main genetic operators:
4. **Mutation:** parts of a candidate solution are randomly changed to create a new candidate solution.
    a. **Crossover:** two candidate solutions are mixed to create a new candidate solution.
    b. **Termination:** the stopping condition for the EA, usually based on runtime or performance.

At its core, an EA works by generating a population of solution candidates or entities, called phenotypes. These entities can be represented in various ways, such as traditional strings or more complex programs. This representation, which is referred to as a genotype, is what the EA manipulates and varies, and it can take many forms, including strings, tree structures, and graphs [10].

Selection is the process of choosing individuals in the current population to be parents for the next generation. A diversity of candidates is required to create a healthy population. Various selection strategies exist, such as tournament-style selection based on fitness values [10]. Elitism is a strategy where the strongest entities remain in the population. The parents of the previous generation become the children of the new generation. This ensures that the best solutions are preserved, which can lead to faster convergence [10].

Genetic algorithms (GA's) are a subtype of evolutionary algorithm which employ techniques such as selection, crossover, and mutation to explore the search space of solutions using a chromosome representation for the population [11]. It is this class of EA that is of interest to us in this work, with the specific GA discussed in section 4.2.2.

3. Literature Review

3.1 Numerical Optimization for Robot Manipulator Motion Planning

A seminal work in using direct optimization for robot manipulation trajectory planning is the CHOMP algorithm, presented at ICRA 2009 by Ratliff et al., from Carnegie Mellon [12]. CHOMP stands for Covariant Hamiltonian Optimization for Motion Planning, and it uses covariant gradient descent to iterate towards an optimal solution. The cost being minimized is composed of two terms, one of which measures the cost of being near an obstacle, and another which optimizes for dynamical traits such as trajectory smoothness and acceleration. The update rule applied at every iteration aims to decrease the cost while retaining trajectory smoothness. Joint limits are handled by projecting illegal joint values back onto the feasible set. In the results presented in the initial paper, CHOMP was implemented on a 6 DOF robotic arm, and was able to solve 85 of the 105 tested problems, with most successful runs completing within 100 iterations, or 3 seconds. Adding random restarts resulting in smooth, collision-free trajectories being found for all 105 test cases. CHOMP remains one of the most commonly used motion planning

algorithms for multi-DOF robotic manipulators. CHOMP's success has led to development of similar optimization-based trajectory planners, including STOMP, a stochastic variant introduced in 2011. STOMP uses a stochastic optimization method which requires no gradient information, and is therefore better able to escape from local minima. Results in a 7-DOF manipulator simulation environment showed that STOMP was able to generate 196/210 successful trajectories, while CHOMP only achieved 149/210.

The first author of CHOMP went on to publish a more recent work that used interior point optimization for motion planning, titled "An Interior Point Method Solving Motion Planning Problems with Narrow Passages" [14]. Published in 2020 by Mainprice, Ratliff, Toussaint and Schaal, this work is the first to present the use of an interior point algorithm for solving complex path planning problems. Its focus is on understanding and exploiting the underlying geometry of the Riemennian manifold on which the obstacle-free workspace exists. The objective functions and constraint terms are developed based on this Riemannian geometry, and can be interpreted as functions of what the authors term "geodesic flow". In simulation of 3-DOF and 6-DOF free-flying robots, their geodesic-attractor objective function was shown to be 91% effective at finding a solution, and 97% effective at avoiding collisions. Comparatively, the optimizer that used a benchmark Euclidean-based objective function was only able to achieve 62% success, and 69% obstacle avoidance. This work is an excellent demonstration of how the problem's properties can be cleverly leveraged to formulate objective functions and constraints that result in much more tractable optimal control problems.

Finally, in 2023, Vanroye et al., presented the FATROP algorithm, which stands for "A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control" [15]. FATROP also uses a primal-dual interior point optimizer, which is inspired by IPopt. However, they exploit the specific structure of the trajectory optimization problem, by formulating their solver to handle the direct multiple shooting approach rather than direct single shooting. Direct single shooting eliminates all but the first state variables by substituting in the system dynamics. By contrast, multiple shooting retains all state variables as decision variables, and represents the system dynamics as equality constraints. This is shown to have better convergence properties, and also allows for an infeasible initialization point. The authors tested their algorithm on a variety of well-known robotics problems, including quadrotor path planning and 7-DOF manipulator motion planning. FATROP was shown to find a solution faster than IPopt for all problems. For the 7-DOF manipulator, IPopt and FATROP each took 38 iterations, but FATROP took only 35.51 ms while IPopt took 136.63.

3.2 NSGA-II

The NSGA-II algorithm first originated in 2002 in the work of Deb et al. [16], and is a seminal work in the field of genetic algorithms. The goal was to create a better and more efficient algorithm for multi-objective optimization. This was achieved by implementing the concepts of elitism and the non-dominated sorting of Pareto frontiers. The work of Ma et al. [17] presents a comprehensive survey on the variations and implementations of the NSGA algorithm. In terms of non-dominated sorting, it states that a solution y is said to dominate a solution x if y is at least as good as x for all objectives, and it is better than x for at least one objective. A solution is said to be non-dominated if there is no x in the population that dominates it. The complete set of solutions are saved to a temporary population T where all non-dominated solutions are found. The non-dominated solutions are regarded as the set B, and a cost value of 1 is assigned. Then the solutions in B are removed from T and non-dominated solutions are found in the reduced set T, which are assigned a cost value of 2. This step is repeated until all the solutions have been assigned a cost value based on their non-dominated levels. NSGA-II is unique for its incorporation of a crowding distance mechanism to sort the Pareto frontiers.

Another version of the NSGA-II algorithm, called R-NSGA-II, is implemented in the work of [18]. This variation uses the concept of multiple initial reference points to discover regions of solutions on the Pareto frontier rather than a single solution. This allows the researcher to choose between various optimal solutions based on preference. This version of NSGA-II is implemented in the work of [19], which consists of trajectory generation for a controller floating space robot. This application is very similar to that which is considered for this project and is used as inspiration for the trajectory generation problem.

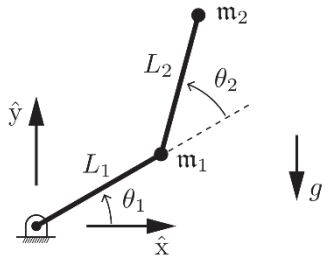
4. Implementation

4.1 Problem Setup

To explore the performance of numerical trajectory optimization techniques in the in the context of a 2-link manipulator, a codebase has been developed to evaluate the performances of each solution based on its feasibility, input energy required, trajectory duration, and input torque requirements. Furthermore, static obstacles have been introduced to ensure that the offline planners are able to converge to a trajectory that avoids any potential collisions. Sections 4.1.1 to 4.1.5 detail the environmental conditions, constraints, and objective functions of the experiments.

4.1.1 Two-link manipulator dynamics

The manipulator used for this comparison is a 2-link arm with 2-DOF along the same plane, essentially constraining the configuration space to 2 dimensions. The robot arm, as defined in Figure A1, is represented as having a point mass at the end of each of the two links. The manipulator was simulated to have the following parameters:



Manipulator Parameters	
L1	2 m
L2	1 m
m1	6 kg
m2	3 kg

Figure 1: 2-link Manipulator Problem Setup

The state variable of the manipulator, X, is defined by the two joint angles at a given point in time, as well as their respective angular velocities, as seen in Equation (4). Additionally, the input torques at each of the two joints is defined similarly as seen in Equation (5). It should be noted that the angular acceleration is intentionally excluded from the state variable because its inclusion would over constrain the optimization problem given that the input torque, angles, and velocities are used to define the angular acceleration. Further details on the calculation of these angular accelerations are included later in this section.

$$X = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \tag{4}$$

$$\tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \tag{5}$$

The primary moments experienced at each of the joints are due to the inertia of the system, as well as the centripetal, Coriolis, and gravitational forces which all vary with time depending on the state of the system. The applied torque may be used to balance the total moment applied at each of the joints to achieve the desired state, as detailed in Equation 3. Furthermore, the $\boldsymbol{M}$ (mass), $\boldsymbol{c}$ (centripetal and Coriolis), and $\boldsymbol{g}$ (gravity) matrices are defined for this specific problem setup as defined in Equations (7) – (9). These expressions are derived from the Euler–Lagrange equations, made by formulating the Lagrangian in terms of the system's kinetic and potential energy at a given instance, assuming that there are no external forces acting on the system and that friction is negligible. Further

details on the specific derivation are detailed in the referenced text [20].

$$\tau = M(\theta)\ddot{\theta} + \underbrace{c(\theta,\dot{\theta}) + g(\theta)}_{h(\theta,\dot{\theta})}, \tag{6}$$

$$M(\theta) = \begin{bmatrix} \mathfrak{m}_1 L_1^2 + \mathfrak{m}_2(L_1^2 + 2L_1L_2\cos\theta_2 + L_2^2) & \mathfrak{m}_2(L_1L_2\cos\theta_2 + L_2^2) \\ \mathfrak{m}_2(L_1L_2\cos\theta_2 + L_2^2) & \mathfrak{m}_2 L_2^2 \end{bmatrix}, \tag{7}$$

$$c(\theta,\dot{\theta}) = \begin{bmatrix} -\mathfrak{m}_2 L_1 L_2 \sin\theta_2(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) \\ \mathfrak{m}_2 L_1 L_2 \dot{\theta}_1^2 \sin\theta_2 \end{bmatrix}, \tag{8}$$

$$g(\theta) = \begin{bmatrix} (\mathfrak{m}_1 + \mathfrak{m}_2)L_1 g\cos\theta_1 + \mathfrak{m}_2 g L_2\cos(\theta_1+\theta_2) \\ \mathfrak{m}_2 g L_2\cos(\theta_1+\theta_2) \end{bmatrix}, \tag{9}$$

4.1.2 Optimal Control Problem Transcription

The torque exerted on the links at each point along the path, based on the angular velocities and positions at that moment, dictates the overall angular acceleration. These accelerations determine the angular velocities and positions of the 2 links for the next time step in a discretized time problem setup, which is the case for the optimizers utilized in the defined problem.

Equation (6) may be rearranged to isolate for the system's angular accelerations, given the torque inputs as well as the angular velocities and positions (system state) as seen in Equation (10). Although varying optimizers may have different approaches in solving for a trajectory, it is the optimized torque that dictates the resulting movement of the system.

$$\ddot{\theta} = M(\theta)^{-1}(\tau - c(\theta,\dot{\theta}) - g(\theta)) \tag{10}$$

With the described discretized problem, it is necessary to constrain the angular positions and velocities of the next time step based on the current angular velocities and accelerations, respectively, as seen in Equation 11, in order to adhere to Newton's laws of motion.

$$\begin{aligned} &\text{For } k = 0 \text{ to } N-1: \\ &\theta_1^{(k+1)} = \theta_1^{(k)} + \Delta t \times \dot{\theta}_1^{(k)} \\ &\theta_2^{(k+1)} = \theta_2^{(k)} + \Delta t \times \dot{\theta}_2^{(k)} \\ &\dot{\theta}_1^{(k+1)} = \dot{\theta}_1^{(k)} + \Delta t \times \ddot{\theta}_1^{(k)} \\ &\dot{\theta}_2^{(k+1)} = \dot{\theta}_2^{(k)} + \Delta t \times \ddot{\theta}_2^{(k)} \end{aligned} \tag{11}$$

4.1.3 Additional Constraints

To ensure that the results produced by the optimization are feasible and realistic, other additional environmental constraints are required. This includes setting magnitude limits to the input torques, as well as the angular positions, velocities, and accelerations. The specific upper and lower limits are detailed in Table 1, and were determined through experimentation.

Table 1: Upper and lower limits of input torque and angular positions, velocities and accelerations

Variable	Lower Bound	Upper Bound
θ_1	0	π/2
θ_2	-π	π
$\dot{\theta}_1, \dot{\theta}_2$	-3 rad/s	3 rad/s
$\ddot{\theta}_1, \ddot{\theta}_2$	-5 rad/s^2	5 rad/s^2
τ_1, τ_2	-500 Nm	500 Nm

Furthermore, to aid with convergence, the total time of the solutions was constrained to non-negative values of time, and with a magnitude of < 4 seconds (determined to be sufficient time for any smooth trajectory in configuration space). The final set of constraints required are those of the boundary conditions. Based on the specific path problem, the starting and end points are defined by the user, which can be converted to joint angles outside of the optimization problem using inverse kinematics. These specified angles determine the boundary conditions for the two joint angles. To allow for a smooth and realistic trajectory to be planned, the angular velocities at the beginning and end of the trajectory are constrained to be 0 rad/s to reduce any jerky initial movements or accidental collisions by the end effector at the goal position.

It should be noted that for all the aforementioned equality constraints, a slight bi-directional tolerance of 1e-4 was provided. This tolerance transforms each equality constraint into 2 inequality constraints, which are by nature easier for optimizers to adhere to without constraint violations and should aid in achieving convergence without significantly worsening the smoothness of the solution.

4.1.4 Objective Functions

An essential component of defining the optimizer configuration is determining an objective cost function. This cost function is utilized by the optimizer to determine the gradient of the solution and progressively converge to a solution that locally minimizes the specified cost function. For the 2-link manipulator, three different objective functions were defined. The first function, as detailed in Equation (12), aims to minimize integral approximation of the input energy required, which is a function of the applied torques and angular velocities. The second equation simply aims to minimize the total time required for the manipulator to move from one position to the other and aims to produce the shortest/fastest feasible path, as detailed in Equation (13). The third function aims to minimize the sum of the torques at all the specified timesteps, as seen Equation (14). Although similar to the first function, the third function primarily aims to minimize the applied torque, regardless of the angular velocities that result from such optimization.

$$\min(E = dt \cdot \sum_{i=0}^{N-1} \tau_i \cdot \dot{\theta}_i) \tag{12}$$

$$\min T \tag{13}$$

$$\min(\check{\tau} = \sum_{i=0}^{N-1} \tau_i) \tag{14}$$

4.1.5 Obstacle Avoidance

Obstacle avoidance was implemented by assuming that each obstacle could be contained within a circular boundary. A series of equality constraints was added to ensure that for each link, the minimum distance between the link and the centre of the obstacle was greater than the radius of the obstacle's boundary. Thus, for an obstacle specified to be centred at location *(x, y)* and with radius *r*, the obstacle avoidance equality constraints for link *i* can be expressed as follows:

$$\min(\, \|(x_i, y_i) - (x, y)\| - r \geq 0 \tag{15}$$

where x_i, y_i can be any Cartesian coordinates that fall on the length of link i.

4.2 Tools

4.2.1 Interior-point Optimization Tools

The Interior-point optimization was integrated and evaluated using a popular python optimization, Casadi [21]. Casadi's framework involves the symbolic representation of the time duration, state, and input variables over the entire trajectory. While adhering to all the defined problem constraints, CasADi's automatic differentiation engine comes into play during optimization iterations to efficiently compute the gradients and Hessians, and progressively converge to a locally optimal solution. Given a specified number of discrete trajectory steps (set to 200 steps), the Casadi library's built-in integration with Interior-point optimization was utilized to optimize the corresponding time stamps for each step, as well as the corresponding input torques that yielded the optimal states for the trajectory.

4.2.2 NSGA-II Optimization Tools

Pymoo [22] is a versatile optimization library in Python designed for multi-objective optimization tasks. It offers a comprehensive set of algorithms, problem definitions, and analysis tools tailored specifically for multi-objective optimization problems. Pymoo enables researchers and practitioners to efficiently solve complex optimization problems with multiple conflicting objectives, allowing them to find trade-off solutions known as Pareto-optimal solutions. With Pymoo, users can easily implement and compare various optimization algorithms, customize their optimization problems, and visualize the results to gain insights into the trade-offs among different objectives. One of the evolutionary optimizers available in Pymoo is the NSGA-II algorithm. As such, Pymoo was used to set up the Genetic Algorithm optimization problem.

4.3 Results

Using the Casadi and Pymoo libraries, the defined manipulator was incorporated such that an optimal discrete trajectory could be determined for a given initial and final cartesian coordinate position. For consistency in the comparisons, the location (2.95, 0.2) was chosen as the initial coordinates, and (1.2, 0.933) as the final position. Inverse kinematics were applied outside of the optimization problem to determine the corresponding angles for both joints. Sections 4.3.1, 4.3.2, and 4.3.3 detail the performances of the optimizer solutions.

4.3.1 Comparison of Objective Functions

Using the interior-point optimization method in Casadi, the problem was set formulated in accordance with the constraints established in Sections 4.1.1, 4.1.2, and 4.1.3. To evaluate the optimizer's ability to adapt to varying target objective functions, 3 experiments were run to minimize input energy, simulation duration, and input torque, respectively. The exact definition of these objective functions is detailed in Section 4.1.4. Figure 2 details the determined trajectories for each of the first two objective functions.

It can be noted that the trajectory for minimizing input energy appears less direct but leverages the force of gravity to do some of the work in moving link 2 so that less input energy is required overall. Contrastingly, the trajectory on the right side of Figure 2 shows the most direct and fastest path that the manipulator could solve for while adhering to the dynamic constraints.

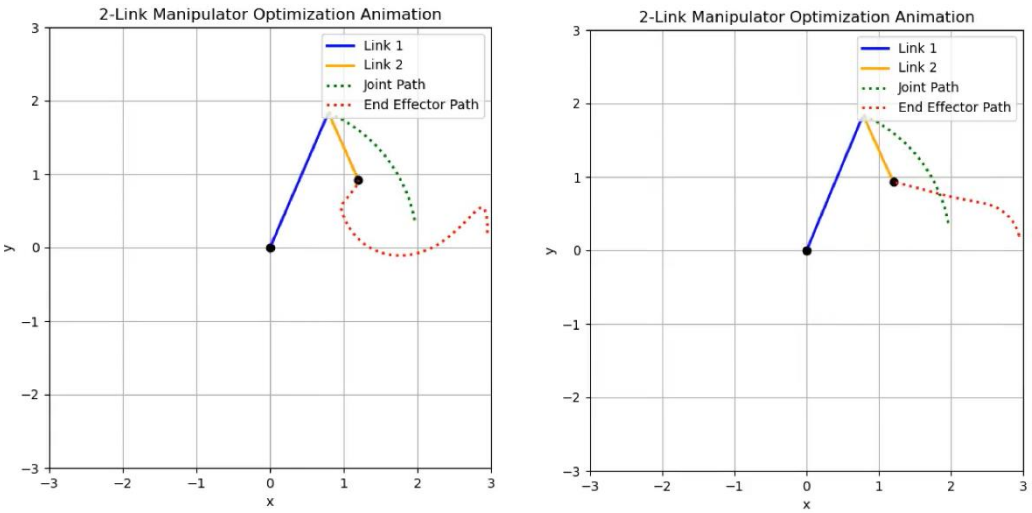


Figure 2: Trajectories obtained using Minimum Energy (left) and Minimum Time (right) Objectives

The two images in Figure 3 demonstrate the trajectory that minimizes input torque at two different time steps (middle and end of the trajectory, respectively). For this solution, it can be noted that as the first link is raised to a 90°, link 2 is allowed to fall with gravity until reaching a state where no torque is required to stay stationary, as seen in the left image of Figure 3. Through iterations of testing, it was determined that the optimizer seemed to stay in this position as long as possible until eventually moving to the final position within the maximum total time constraint, as seen on the right image of Figure 3. This behaviour yielded difficulties in convergence of the optimizer, and the final permitted iteration (iteration #999) of the optimizer solution produced the results seen in Figure 3 and Table 2.

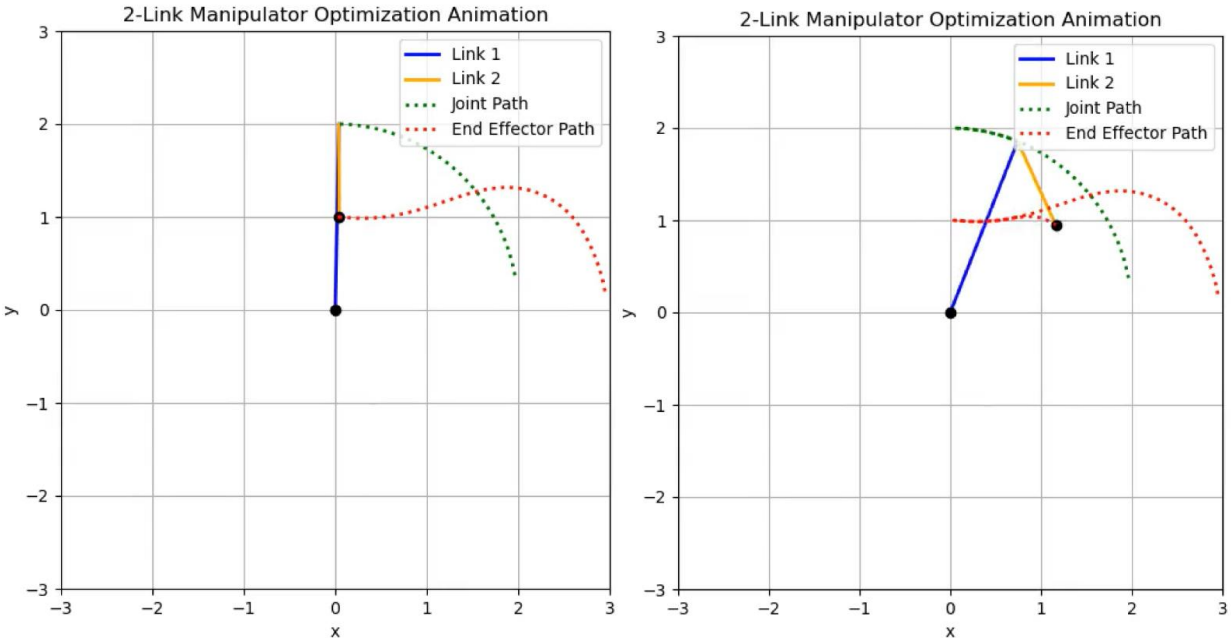


Figure 3: Minimizing Input Torque at Intermediate (Left) and Final (Right) Time Step

The results detailed in Table 2 indicate the number of iterations and computational time required for each of the three defined experiments, as well as their performance in the three defined metrics. The primary takeaway from Table 2 is that each of the three trials was able to successfully minimize their respective objective function better than the other two trials.

Table 2: Performances of the three trials with varying objective functions.

Objective Function	Iterations	CPU Time	Input Energy	Simulation Duration	Sum of Torques
Minimize Input Energy	51	2.782 s	161.7 J	3.407 s	33.04 kNm
Minimize Duration	60	2.419 s	192.5 J	1.247 s	36.33 kNm
Minimize Input Torque	999 (max)	40.26 s	234.0 J	3.99 s	13.03 kNm

Further insight into the state of the manipulator and the torque inputs throughout the trajectory may be found in Figure 4. These results correspond to the case where the input energy was being minimized. It can be seen that the angular position and velocities appear smooth throughout the trajectory to ensure that any rapid jerks that could harm the equipment or spike the current are minimized. Furthermore, all the state and input values are seen to adhere to the upper and lower limits defined in Section 4.1.3, while also approaching angular velocities of 0 rad/s at the beginning and end of the trajectory. Similar Figures for the second and third objective functions may be found in Appendix A.

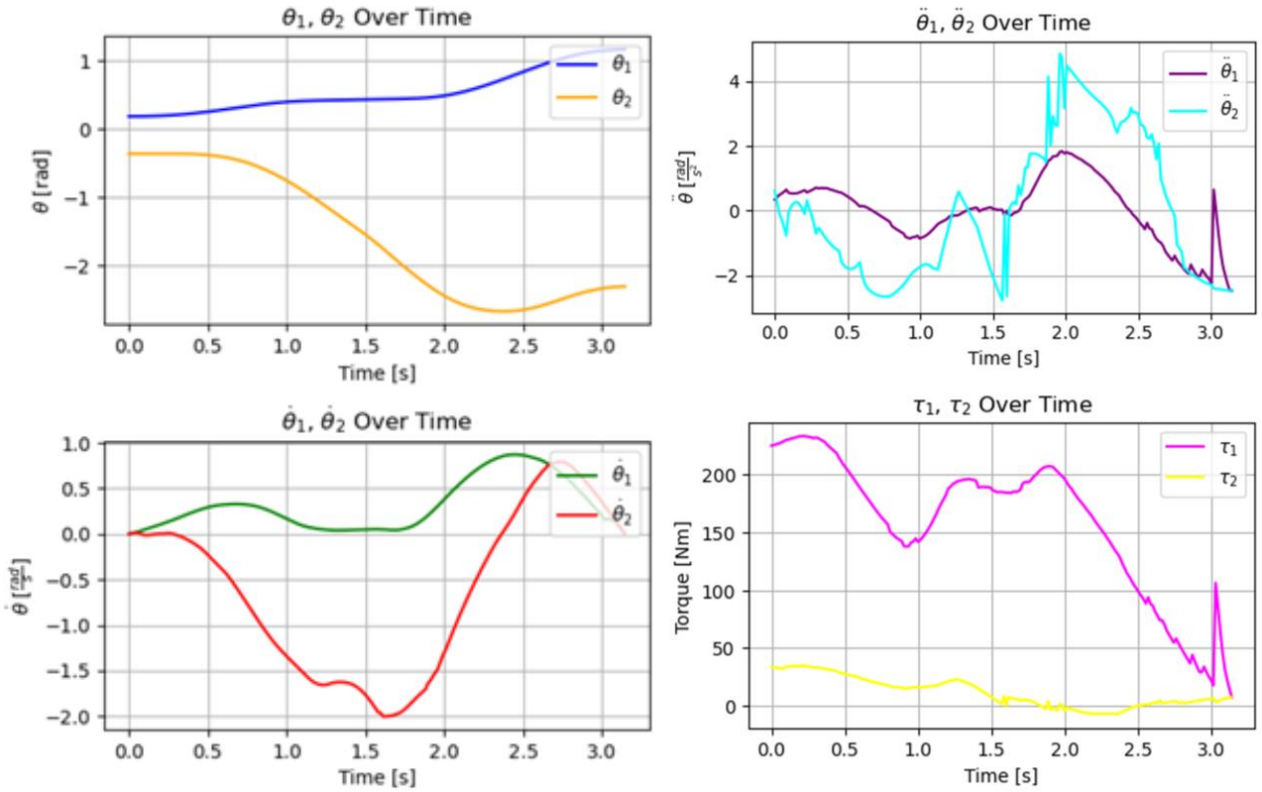


Figure 4: State and Torque Inputs When Minimizing Input Energy

4.3.2 Obstacle Avoidance

A singular circular obstacle was placed in the path of the end effector trajectory, and the minimum-energy optimization was performed with the obstacle avoidance feature enabled. It can be observed in Figure that the optimizer is able to successfully avoid the obstacle once the collision checking constraints are enabled.

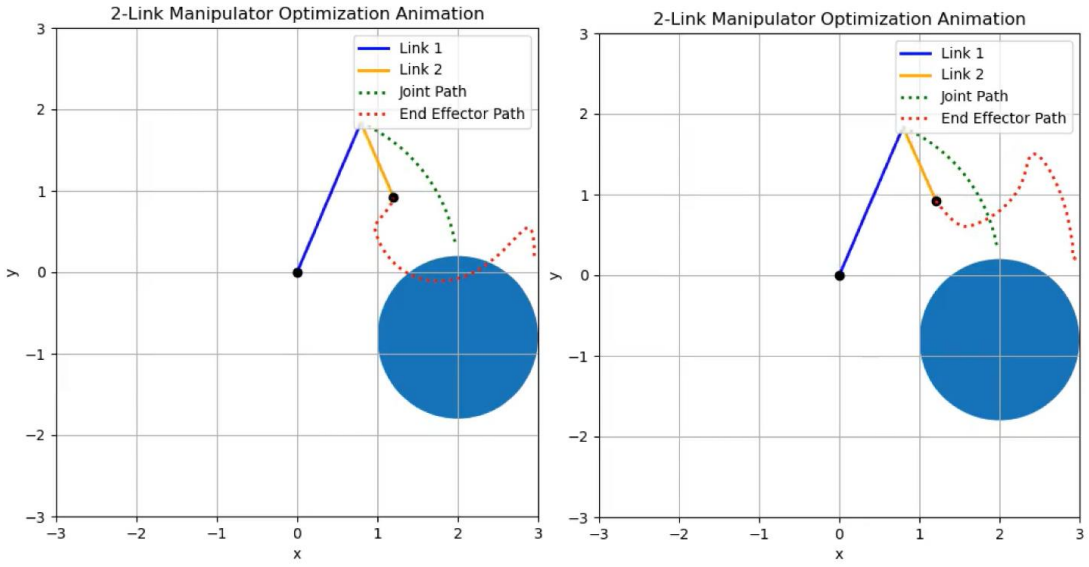


Figure 5: Obstacle Avoidance disabled (left) and enabled (right)

4.3.3 Genetic Algorithm Performance

The path planning problem was also implemented using the NSGA-II solver in Pymoo. In this case, the algorithm fails to converge since it fails to satisfy the problem's constraints. Upon analysis of the simulation data, it can be observed that although the algorithm comes close to finding dynamically feasible paths, it struggles to fully converge. It is worth noting that the results presented are using scaffolding of the initial population. This means that some naïve trajectories are generated to sample for the initial population to guide and constrain the search space for the algorithm. However, this was not enough to ensure the convergence of the algorithm.

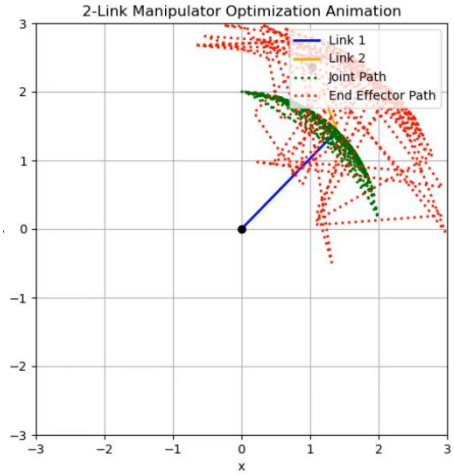


Figure 6: Genetic Algorithm trajectory

5. Open Problems

This project has shown that although genetic algorithms may be good at multi-objective optimization, they can suffer from convergence issues when dealing with too many interrelated constraints. In this case, the dynamical constraints imposed at each timestep posed a problem for convergence. This can be attributed to the random nature of the GA since it is difficult to satisfy all the constraints purely randomly and in a logical manner. This is where a traditional algorithm, such is IPopt, has an advantage since it uses a structured gradient descent approach to satisfying the constraints. Eventually, the results showed that a loss of diversity in the population occurs, preventing the algorithm from achieving a feasible solution. In theory, if a feasible solution was obtained, or a set of feasible solutions, then the algorithm would naturally smooth the path of the joint trajectories through the objective functions.

A few different approaches could be explored in the future to address this problem and make it more suitable for the Genetic Algorithm solver. Firstly, a similar problem setup to the trajectory generation for a floating space robot example in [19] could be implemented, where the trajectory at each timestep is implemented individually. This would reduce the complexity of the problem since dynamic constraints would only need to be satisfied for adjacent timesteps. Secondly, an alternative trajectory formulation, such as polynomial trajectory encoding, could be considered. This would provide a more compact representation of the trajectory which may be more easily handled by the solver. Finally, a smarter scaffolding method could be implemented to begin optimization with a diverse population of feasible trajectories. This could also be done by not only creating linearly interpolated trajectories (implemented in this project), but also naïve polynomial trajectories. Furthermore, encoding the features (such as curvature) of these trajectories rather than just the joint angles at individual timesteps may help with creating more diverse solutions using the evolutionary operators.

Further investigation into determining the ideal optimizer for real-life optimization tasks includes investigation into online planners that can consider dynamic obstacles. Additionally, a more realistic problem setup would include considerations for the mass of the links, the friction loss, and any external disturbances that may be affecting the manipulator's dynamics.

6. Conclusion

The defined project and experiments were tailored towards comparing the feasibility and capabilities of a classical optimization approach with a genetic algorithm optimizer in the context of a 2 link, 2-DOF manipulator. With this project, we were able to implement an IPopt optimizer, through CasAdi, and an NSGA-II optimizer, through Pymoo, in order to minimize input energy, time, and torque magnitude to the manipulator. The traditional IPopt optimizer was able to converge to generate trajectories to minimize the objectives, for both with and without obstacles. However, the NSGA-II algorithm struggled to converge and satisfy all of the constraints for the entire trajectory. This suggests that other encodings for the trajectory optimization problem may be needed in order to use NSGA-II for trajectory optimization.

7. References

[1] R. Tedrake, “Robotic Manipulation” in Perception, Planning, and Control. 2023. http://manipulation.mit.edu

[2] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065), San Francisco, CA, USA, 2000, pp. 995-1001 vol.2

[3] H. J. Sussmann and J. C. Willems, "300 years of optimal control: from the brachystochrone to the maximum principle," in IEEE Control Systems Magazine, vol. 17, no. 3, pp. 32-44, June 1997, doi: 10.1109/37.588098.

[4] A. E. Bryson, "Optimal control-1950 to 1985," in IEEE Control Systems Magazine, vol. 16, no. 3, pp. 26-33, June 1996, doi: 10.1109/37.506395

[5] E. Todorov. "Optimal control theory." 2006. University of California, San Diego.

[6] A. Lal. “Interior-point Method for NLP”. 2021. Cornell University.

[7] A. Singh. “Primal-Dual Interior-Point Methods”. Carnegie Mellon.

[8] Wächter, A., Biegler, L. “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming”. Mathematical Programming. 106, 25–57 (2006).

[10] A. N. Sloss and S. Gustafson, “2019 evolutionary algorithms review,” Genetic Programming Theory and Practice XVII, pp. 307–344, 2020.

[11] Katoch, S., Chauhan, S.S. & Kumar, V. “A review on genetic algorithm: past, present, and future.” Multimed Tools Appl 80, 8091–8126 (2021).

[12] N. Ratliff, M. Zucker, J. A. Bagnell and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," 2009 IEEE International Conference on Robotics and Automation, Kobe, Japan, 2009, pp. 489-494

[13] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 2011, pp. 4569-4574

[14] J. Mainprice, N. Ratliff, M. Toussaint and S. Schaal, "An Interior Point Method Solving Motion Planning Problems with Narrow Passages," 2020 29th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN), Naples, Italy, 2020, pp. 547-552

[15] Vanroye, Lander et al. “FATROP: A Fast Constrained Optimal Control Problem Solver for Robot Trajectory Optimization and Control.” 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2023): 10036-10043.

[16] K. Deb, A. Pratap, S. Agarwal and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. in IEEE Transactions on Evolutionary Computation, 2002.

[17] Ma, H., Zhang, Y., Sun, S. et al. “A comprehensive survey on NSGA-II for multi-objective optimization and applications”. Artif Intell Rev 56, 15217–15270, 2023.

[18] Huang, Y., Fei, M. “Motion Planning of Robot Manipulator Based on Improved NSGA-II”. Int. J. Control Autom. Syst. 16, 1878–1886, 2018.

[19] Seddaoui, Asma & Saaj, Chakravarthini. “Collision-free optimal trajectory generation for a space robot using genetic algorithm”. Acta Astronautica, 2021. 179. 311-321.

[20] K. M. Lynch and F. C. Park, “Modern Robotics: Mechanics, Planning, and Control”. Cambridge: University Press, 2017.

[21] “Casadi,” CasADi, https://web.casadi.org/ (accessed Apr. 17, 2024).

[22] “Multi-objective optimization in python,” Pymoo, https://pymoo.org/index.html (accessed Apr. 18, 2024).

7. Appendix

Figures A1 and A2 demonstrate the full state and torque inputs of the trajectories for the Interior-point optimization method solution of the second and third objective functions which minimize the simulation duration and input torque, respectively.

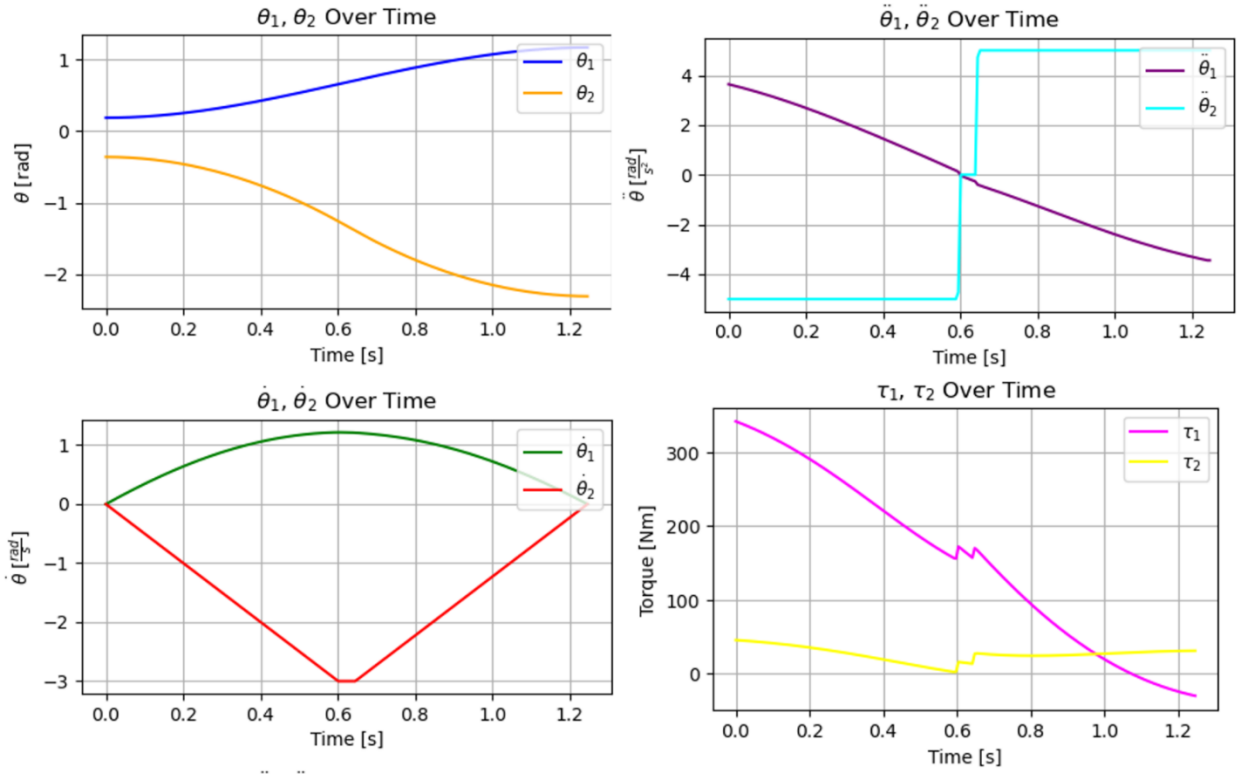


Figure A1: State and Torque Inputs When Minimizing Simulation Duration

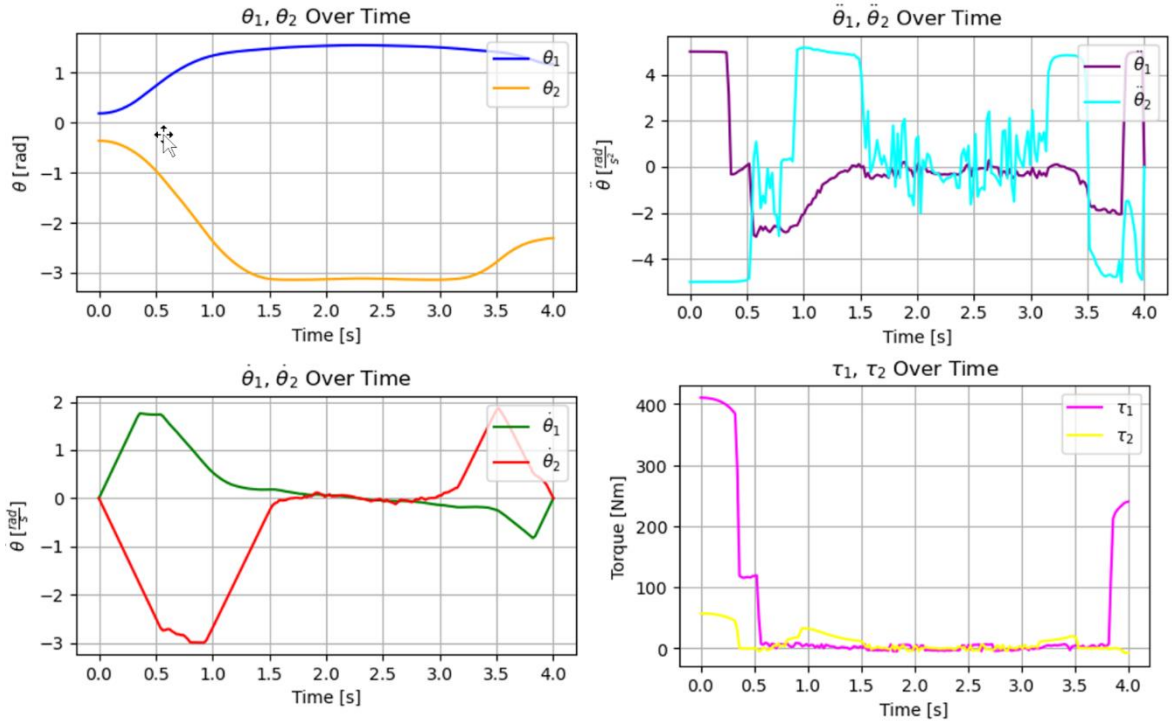


Figure A2: State and Torque Inputs When Minimizing Input Torque